

# CS 486/686

# Neural Networks and Learned Embeddings

Yuntian Deng

---

Lecture 18

From fixed features to learned vector representations

# Recorded lecture (asynchronous)



This lecture is **pre-recorded** - I'm away at **ICML** this week, so there's no in-person class today. Watch at your own pace.



**Due today (Thu Jul 9):** CS686 project proposal. **Assignment 2** is now due **Thu Jul 16**.



Questions? Post on **Piazza** - the TAs are available, and I'll follow up when I'm back.

# What you should take away

One thread runs through the whole lecture: useful vectors can be **learned**.

## **build**

Explain why hidden layers need nonlinearities.

## **represent**

Explain embeddings as learned vectors, including contextual ones.

## **inspect**

Use projection, neighbors, and clusters to reason about embedding spaces.

The later examples are applications of the same idea: learned spaces become interfaces.

## **Act 1: learn the features**

L17 trained models on fixed input vectors.

Today, the model also learns the vector representation.

# The problem with fixed features

L17 assumed the useful input vector already exists.

## pixels

Where is the cat ear?

## word IDs

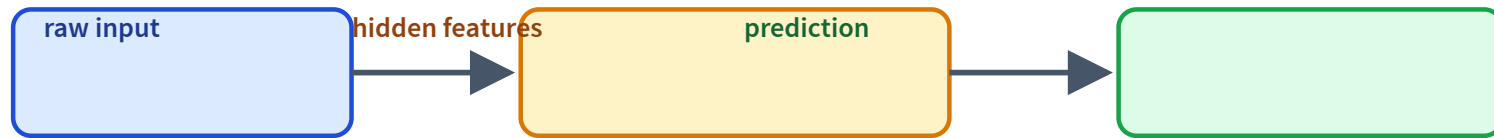
Why is dog  
close to puppy?

## screenshots

Which region  
is a button?

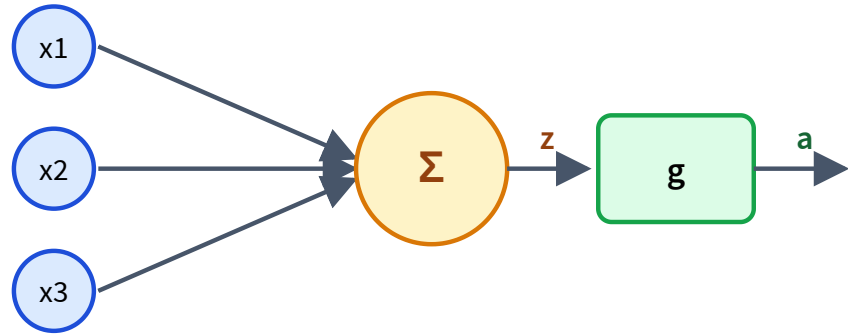
Modern ML learns the features too.

# Neural nets learn representations



The hidden layer is not just computation: it is a learned view of the input.

# A neuron is a feature detector



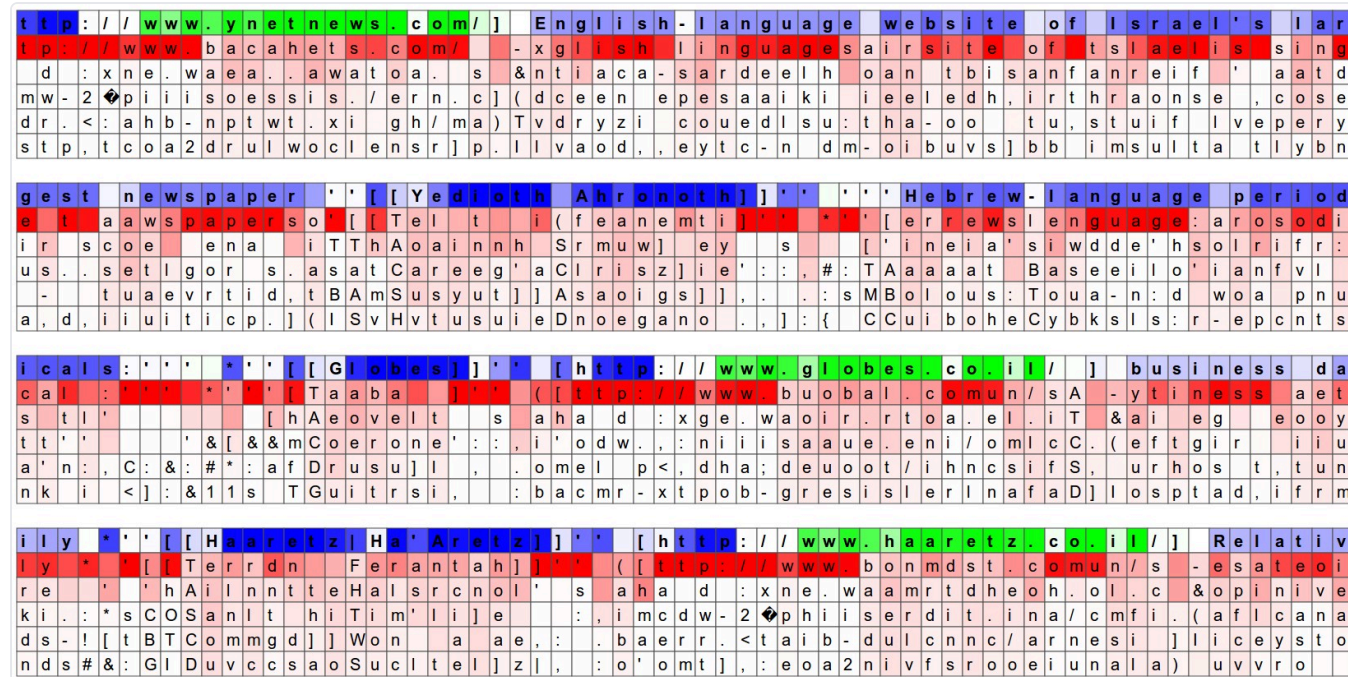
inputs  $x_1, x_2, x_3$

weighted sum:  $z = w^\top x + b$

activation:  $a = g(z)$

# A real learned neuron

Each character is colored by **one** neuron's activation in a character-level RNN.

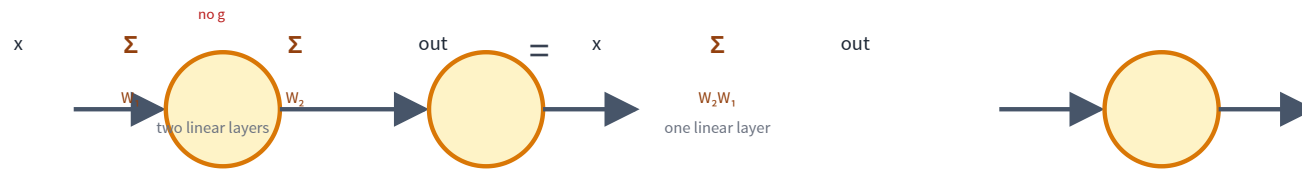


Nobody programmed it — this neuron learned to fire inside URLs.

Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks," 2015.

# Why add a nonlinearity?

Two linear layers in a row collapse into one:

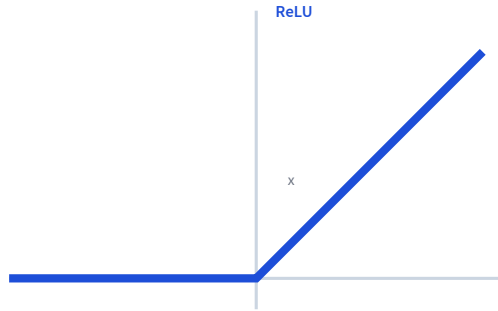


$$W_2(W_1x) = (W_2W_1)x$$

Depth alone adds nothing — we need a nonlinearity between the layers.

# The fix: a nonlinearity

Insert  $g$  between layers:  $h = g(W_1x)$ ,  $\hat{y} = W_2h$ .



$$g(x) = \max(0, x)$$

The bend is what lets stacked layers represent nonlinear functions.

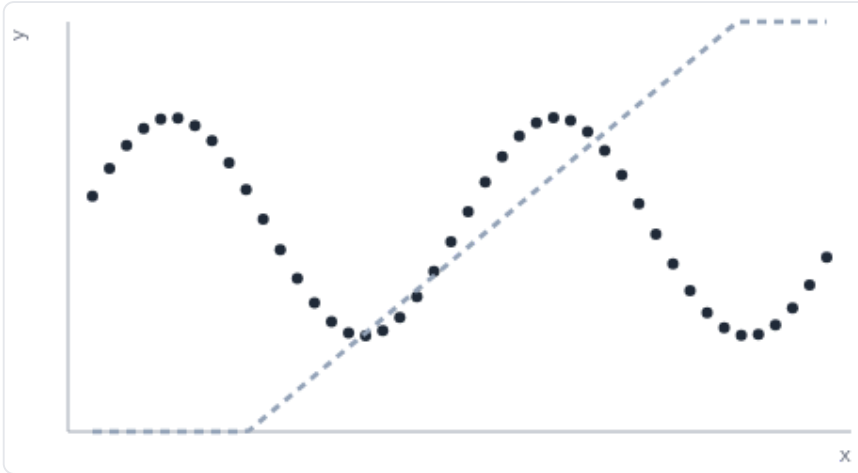
# Playground: does nonlinearity matter? LIVE

Same net, activation off vs. on — try it on the live slides.

```
nn.Sequential(nn.Linear(1,12), nn.Tanh(), nn.Linear(12,1)) · target:  $y = \sin(2x)$ 
```

activation: OFF

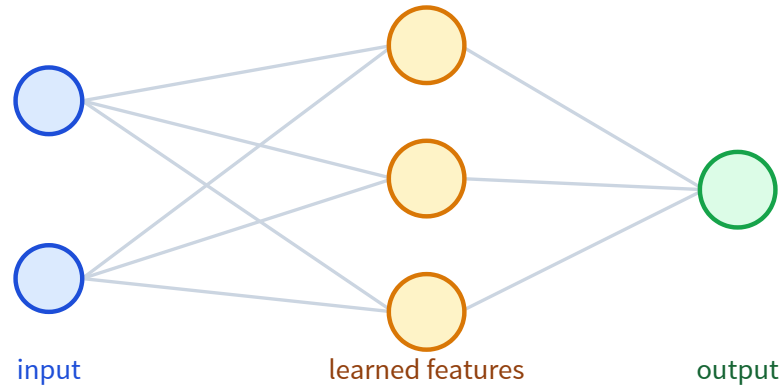
Train



activation: **OFF (linear)**  
untrained — press **Train**

Same net, same data. Activation OFF: stacked linear layers stay a line. ON: the layer bends to fit the wave. That bend is what nonlinearity buys.

# One hidden layer = learned feature map

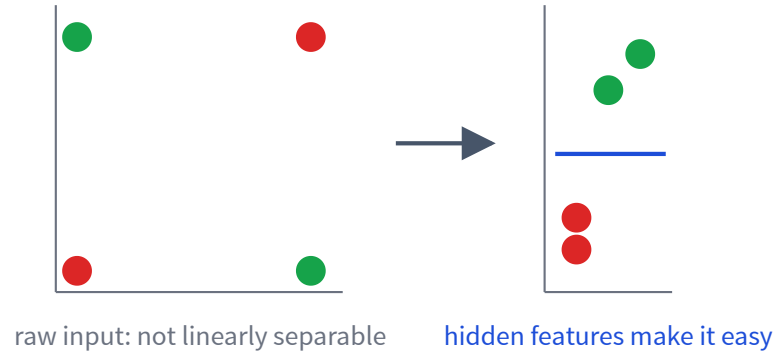


Each hidden unit learns a detector.

The output layer  
combines detectors.

This is how fixed features  
become learned features.

# XOR: why hidden features help



A hidden layer can transform the space.

Then a simple output layer can solve the task.

# Playground: train a net on XOR LIVE

Train a tiny net on XOR — try it on the live slides.

```
nn.Sequential(nn.Linear(2,2), nn.Tanh(), nn.Linear(2,1)) · task: XOR
```

**Train**

activation: OFF



activation: **OFF**  
untrained — press **Train**  
○ class 0    ■ class 1

Green/red = the net's decision surface; markers are the true labels. Turn the activation OFF and the 2-2-1 net becomes linear — it cannot separate XOR no matter how long it trains.

## An MLP: a bigger model, same loop

$$h = g(W_1x + b_1)$$

$$\hat{y} = \text{softmax}(W_2h + b_2)$$

Every operation is differentiable, so L17's training loop applies unchanged.

Quick check: what changed from logistic regression?

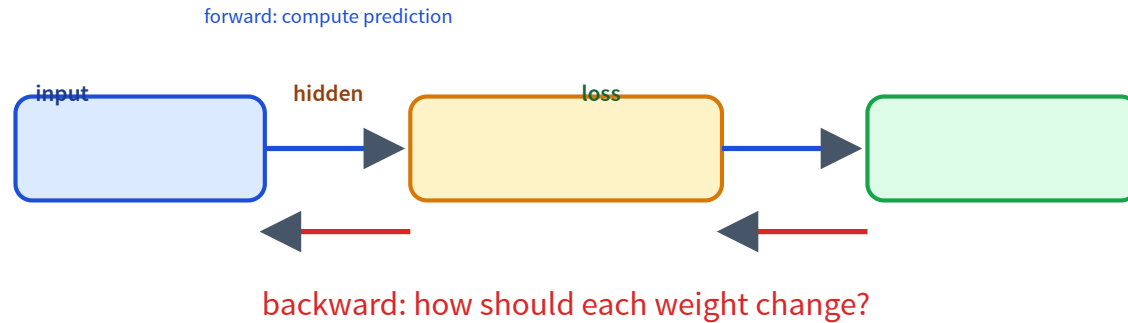
Only the function class, not the optimization recipe.

# The code barely changes

```
model = nn.Sequential(  
    nn.Linear(d_in, 64),  
    nn.ReLU(),  
    nn.Linear(64, n_classes),  
)  
  
loss = loss_fn(model(x), y)  
loss.backward()  
optimizer.step()
```

The function is richer; the training pattern is the same.

# Backpropagation assigns credit



Mechanically, this is the chain rule applied through the computation graph.

# What do hidden layers learn?



Deep models compose simple features into useful abstractions.

# CNNs learn visual features



shallow: curves



middle: parts



deep: objects



one deep neuron = a "ball" detector

Feature visualizations (what each neuron responds to most).

Olah et al., "Feature Visualization," Distill 2017 (CC-BY).

**Local filters** detect small patterns.

Shallow: edges & curves.

Deeper: object parts.

Deepest: whole objects.

Later: VLMs reuse  
image encoders (L23).

## **Act 2: embeddings give symbols geometry**

The same representation idea applies  
when the input is not pixels, but tokens.

A word ID is just a label; an embedding is a learned location in a vector space.

# Word IDs have no geometry

IDs are arbitrary labels:

cat = 17, the  
= 18, dog = 932

id 17 (cat) is no closer to id  
18 (the) than to id 932 (dog)  
— adjacency means nothing.

An embedding gives each a vector,  
where **distance = similarity**:



Similar meaning → nearby vectors (cat close to dog; "the" is off on its own).

# An embedding table is a learnable lookup

It is just a **lookup table** (a dictionary): a token's **id** selects its **row**.

| token | id  | row = learned vector      |
|-------|-----|---------------------------|
| cat   | 17  | [0.22, -0.71, 0.10, ...]  |
| dog   | 932 | [0.19, -0.66, 0.12, ...]  |
| car   | 40  | [-0.80, 0.31, -0.40, ...] |

`emb[17]` → the "cat" row

```
emb = nn.Embedding(  
    num_embeddings=vocab_size,  
    embedding_dim=128,  
)
```

Unlike a fixed dictionary, the rows are **parameters**: gradient descent updates them during training.

# How does an embedding learn?

Words in the **same contexts** get pulled together:

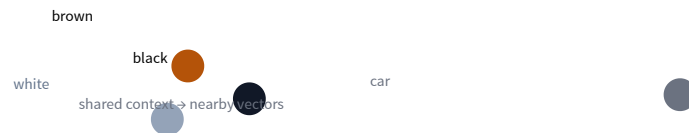
I have a \_\_\_\_ cat is usually a color.

**brown**

**black**

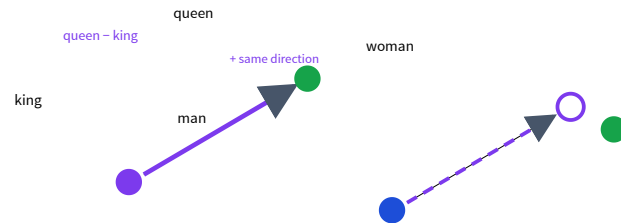
**white**

To fill the same blanks, the model gives these color words **similar embeddings**.



# Directions can carry meaning

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$



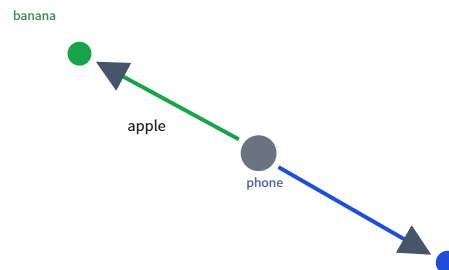
Add the "royal" direction to **man** and you land *near* woman (word2vec, GloVe).

Today this is learned **implicitly inside LLMs**;  
contextual vectors rarely do clean arithmetic.

# One word can mean two things

"I ate an **apple**" → **fruit**

"**Apple** released a  
phone" → **company**



A single fixed vector for "apple" cannot be both.

Modern models embed the **whole sentence**, so a word gets a different **contextual** vector each time.

This creates the L19 problem: context has to move information between token vectors.

## Act 3: inspect learned spaces

Once inputs become vectors, we can ask geometric questions.

**project**

Can I see the space?

**neighbors**

What is close  
to what?

**cluster**

What groups exist?

# Inspect embeddings by projecting them LIVE

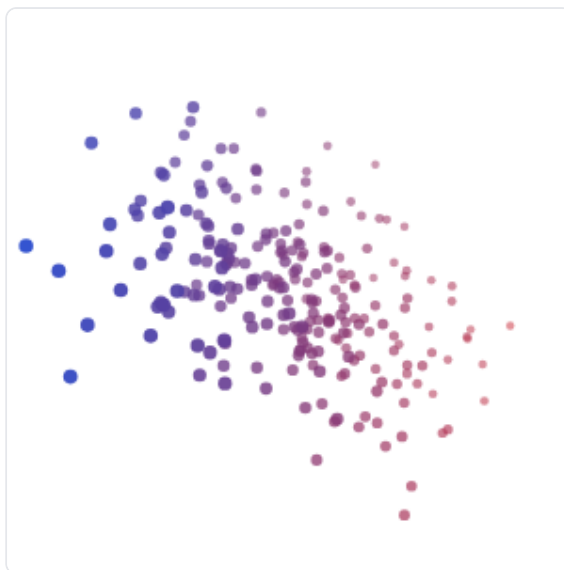
PCA keeps the directions of most spread — drag the cloud, then flatten it.

Drag the 3D cloud to rotate. PCA finds the flat plane of most spread and drops the thin 3rd direction.

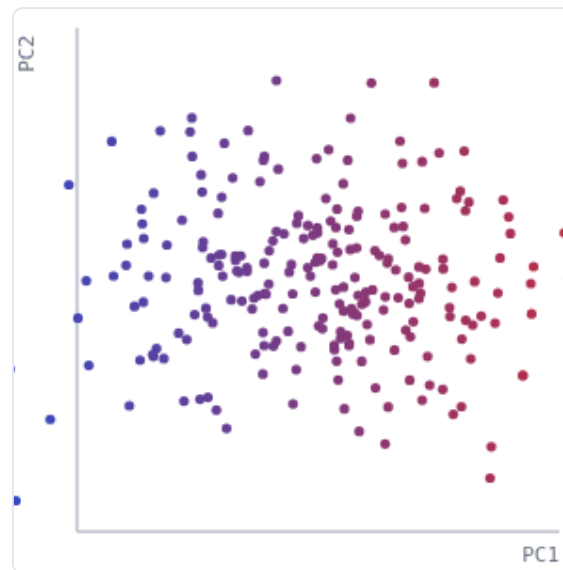
**Flatten to 2D**

**Reseed**

3D data (drag to rotate)



PCA → 2D



# PCA on real data: a million chats

LIVE DATA

Real WildChat conversations (1536-dim) projected to 2D — hover to read, click to open.

Each dot is a real conversation — a 1536-dim vector (OpenAI text-embedding-3-small) projected to 2D with PCA.

Zoom +

Zoom −

Reset view

[Explore it live at wildvisualizer.com](https://wildvisualizer.com) ↗



1428 real conversations — hover to read, click to open.

# Playground: embed real words

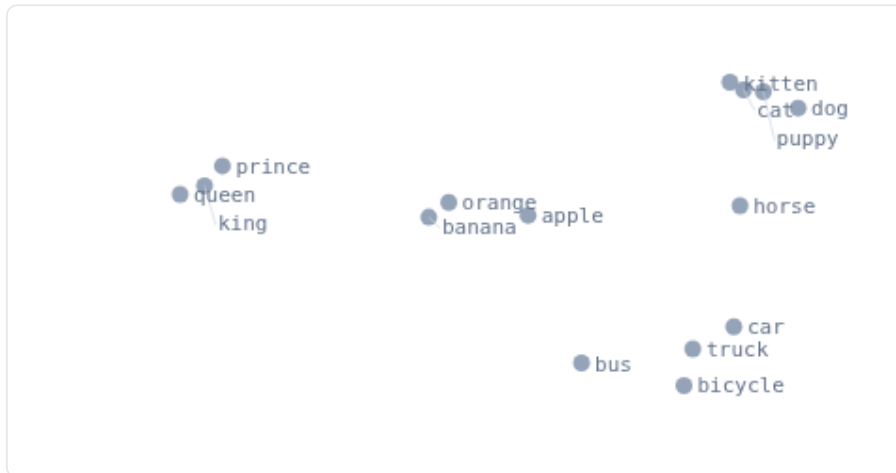
LIVE MODEL

Type words or sentences; a real model embeds them — try it on the live slides.

dog, puppy, cat, kitten, horse, car, truck, bus, bicycle, apple, banana, orange, king, queen, prince

Embed

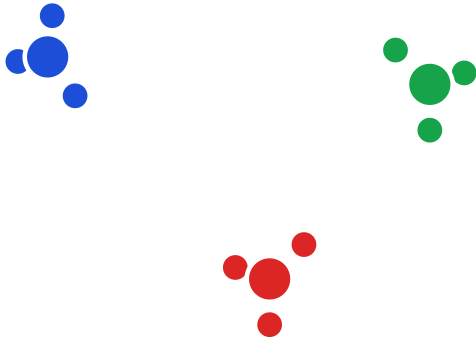
Reset words



click a point to see its  
nearest neighbors

Type any words or short sentences. MiniLM computes a contextual embedding (not a fixed lookup), then PCA squeezes it to 2D. Similar meanings land near each other.

# Cluster embeddings with k-means

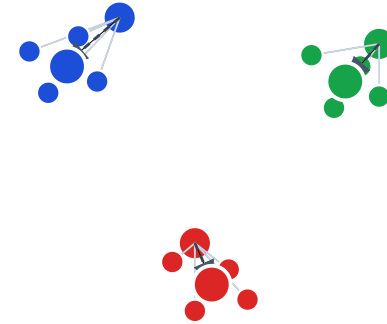


After embeddings exist, k-means becomes a way to summarize the space with prototypes.

- inspect datasets
- find groups or duplicates
- build codebook intuition: replace many vectors with a few representative centers

# The k-means algorithm

1. Choose  $k$ ; initialize  $k$  centroids (e.g. random data points).
2. Repeat until assignments stop changing:
  - a. **Assign** — put each point with its nearest centroid.
  - b. **Update** — move each centroid to the mean of its points.



Converges to a **local** optimum — depends on initialization, so run a few times and keep the best.

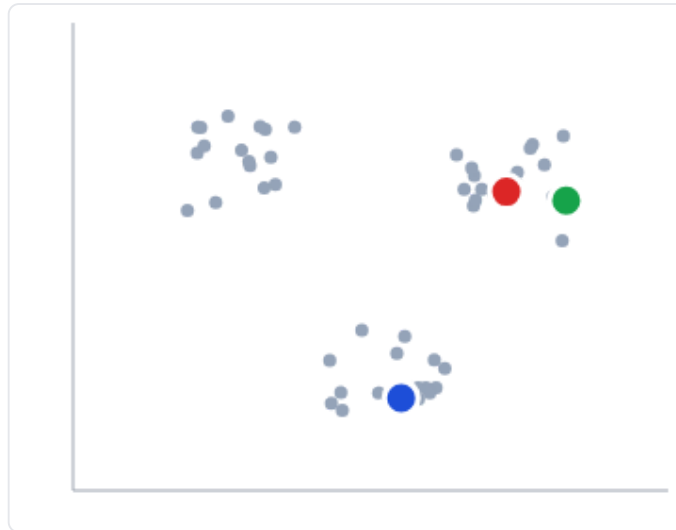
# Playground: k-means, step by step LIVE

Step through assign and update — try it on the live slides.

Step

Run

Reseed



iteration 0  
centroids placed

Each step: assign every point to its nearest centroid, then move each centroid to its members' mean. Repeat until nothing moves.

## **Act 4: learned spaces become interfaces**

So far, we used embeddings for individual points, neighbors, and clusters.

Next: use learned spaces to compare distributions, compress data, and align modalities.

# How do you grade an image generator?

Prompt "a dog" — every sample is a **different** valid dog.

**dog A**

fluffy, sitting

**dog B**

running, brown

**dog C**

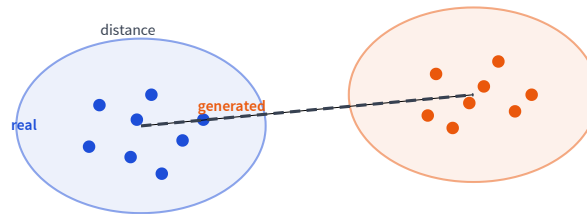
puppy, close-up

There is no single ground-truth image to compare against.

So compare the whole **distribution** of outputs to real images.

# Measuring image quality: FID

Embed real and generated images with an Inception network, then compare the two **distributions** in feature space.



FID = distance between the two feature distributions  
— lower means the generations look more real.

# How do you grade a story generator?

Ask for a short story — there are **countless** good ones.

## story A

a lost dog  
finds home

## story B

a robot learns  
to paint

## story C

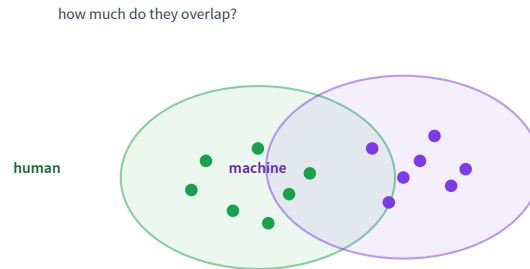
two friends  
and a map

No ground-truth story to score against.

So compare machine stories to **human** stories as distributions.

# Measuring text quality: MAUVE

Embed human and machine text in a language model's space and compare the two distributions.



MAUVE summarizes how much the two distributions overlap — higher means more human-like.

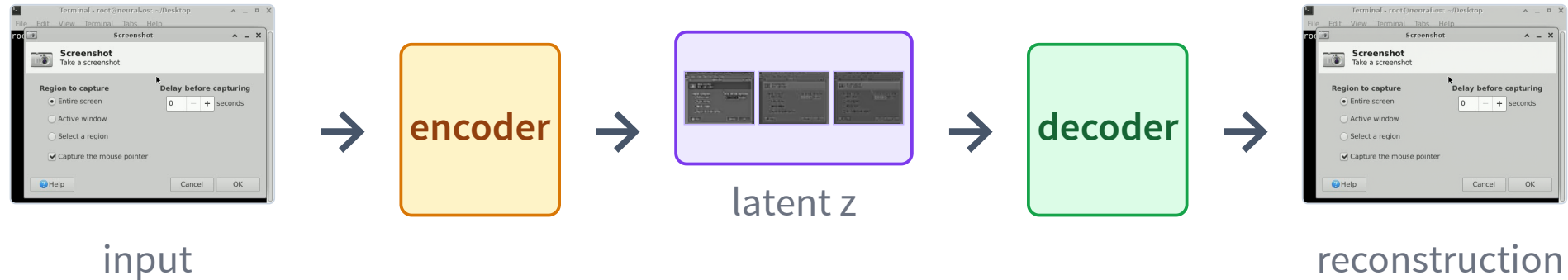
# Autoencoders learn compression

Another use of learned spaces: store what matters in a smaller latent vector.



Latent diffusion will generate in a compressed latent space.

# A real autoencoder: NeuralOS

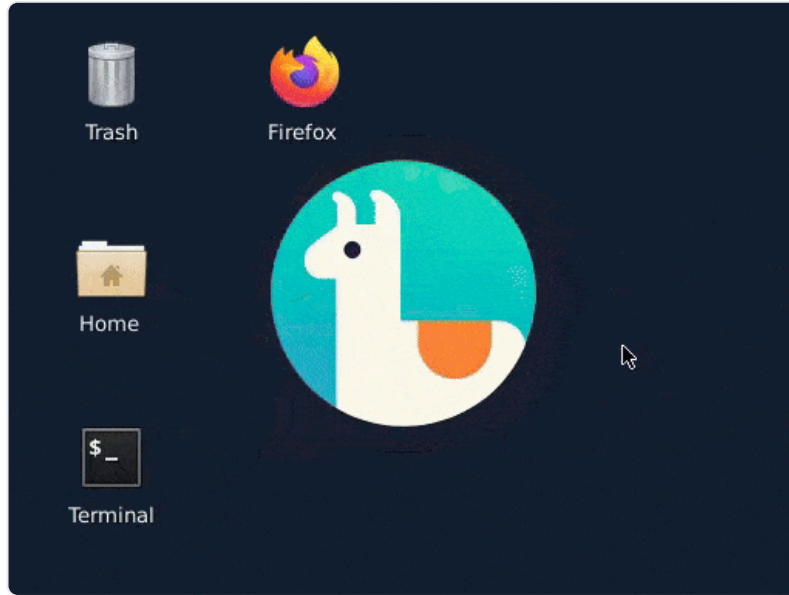


The decode looks the same as the input — the small latent kept everything that matters.

This makes the latent a useful interface: compact enough to model, rich enough to reconstruct.

Autoencoder from NeuralOS ([neural-os.com](http://neural-os.com)).

# Why compress? To make generation feasible



neural-os.com

Generating in pixel space is expensive.

$$512 \times 384 = 196,608 \text{ pixels}$$

$$\rightarrow 64 \times 48 = 3,072 \text{ pixels}$$

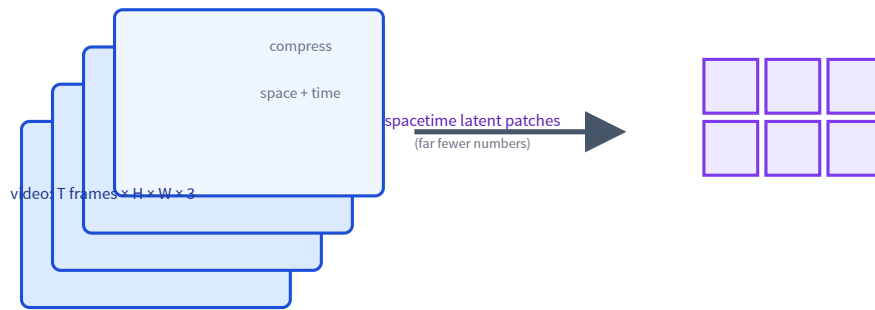
**$\approx 64\times$  fewer pixels to generate.**

NeuralOS predicts the next frame in this small latent — a neural operating system.

Same story as embeddings: do hard work in the learned space, not the raw space.

# Video: compress space and time

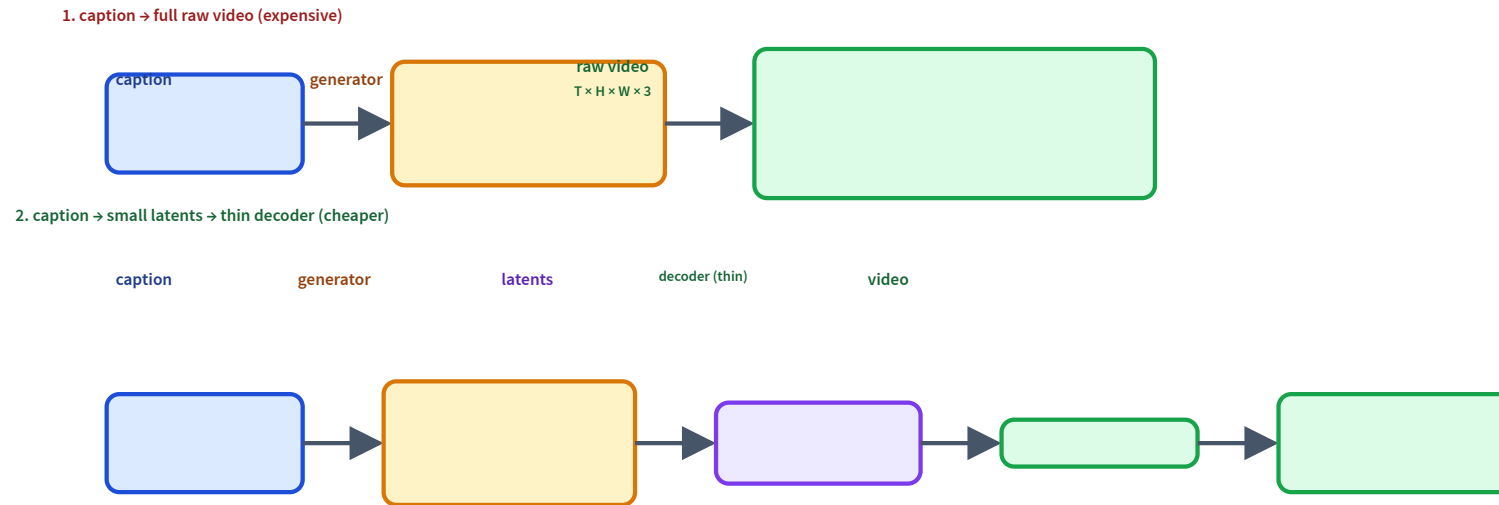
A video is many frames, so pixel cost explodes — compress the time axis too.



Sora compresses space and time into  
spacetime latent patches, then generates there.

OpenAI, "Video generation models as world simulators," 2024.

# Two ways to generate video

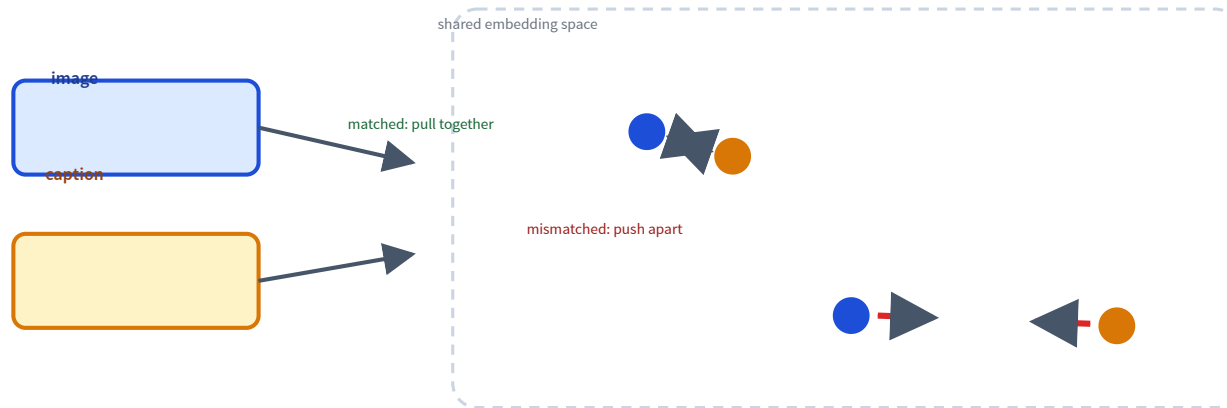


Model the small latents, not the pixels; a thin decoder maps them back to video.

# Contrastive learning aligns modalities

Put images and their captions into one shared vector space.

A final interface: two modalities become comparable by distance.

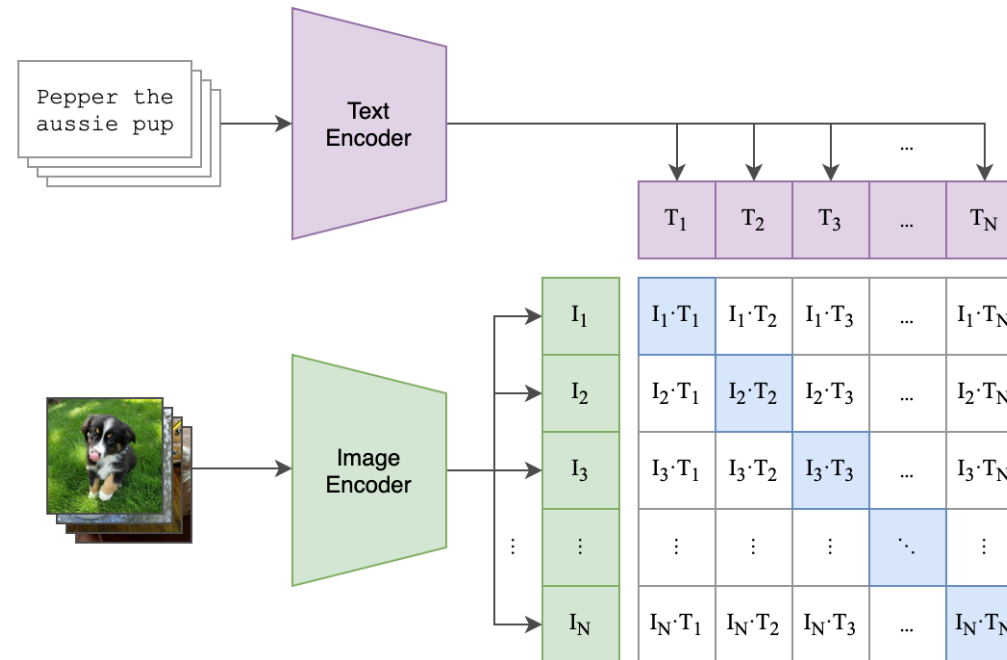


A dog photo lands near "a dog" and a car photo near "a car" — but the dog photo stays far from "a car".

# CLIP: contrastive pre-training

In each batch, every image is compared with every caption; the correct pairs are the diagonal.

(1) Contrastive pre-training



Across a batch, the matching image-text pairs  
(the diagonal) are trained to score highest.

# What a shared image-text space unlocks

## zero-shot classification

score an image against prompts like "a photo of a {label}" — no task-specific training

## retrieval

search images with text (and vice versa) by nearest vectors in the shared space

This shared space is the bridge to vision-language models (L23).

# Why this matters for LLMs

Returning to language: L18 gives the vector objects that L19 will move around.

**tokens**

become embeddings

**attention**

moves information  
between  
embeddings

**transformers**

scale this up

# Carry this forward

| From L17              | L18 adds                | Needed for L19                  |
|-----------------------|-------------------------|---------------------------------|
| gradient training     | hidden representations  | deep sequence models            |
| fixed feature vectors | learned embeddings      | tokens as vectors               |
| loss curves           | PCA, neighbors, k-means | interpreting attention behavior |

The through-line: learn a useful space, then compute inside it.

# Recap

- ✓ Nonlinear hidden layers let neural nets learn **features**, not just predictions.
- ✓ Backprop assigns credit through the graph with the chain rule.
- ✓ **Embeddings** are learned vectors; contextual embeddings depend on the sentence.
- ✓ PCA, t-SNE/UMAP, neighbors, and k-means help inspect embedding spaces.
- ✓ FID, MAUVE, autoencoders, and CLIP show learned spaces acting as interfaces.

## Next question

A sentence is a sequence of contextual embeddings.

How should each word look at the other words?

L19: sequence models, attention, and transformers.